An Empirical Evaluation of Zero-Shot GPT-4o-mini for Java Unit-Test Generation

Le The Khang, Huynh Cao Phuoc, Pham An Khang, Nguyen Thi Nhu Y, and Do Long Vy

Software Testing and Research-Based Learning (SE1944)
FPT University HCMC, Vietnam

Abstract. Large language models can generate unit tests directly from source code, but a successful API response does not by itself establish that the resulting suite compiles, passes, or detects faults. This study evaluates zero-shot GPT-4o-mini unit-test generation for 63 Java functions from a local HumanEval-Java transformation. We measure executable-suite rate, branch coverage, mutation score, and paired differences against retained EvoSuite test suites measured under 1-, 3-, and 5-minute budgets. The generation service returned content for all 63 targets, and one repair pass was attempted where needed; however, only 14 generated suites executed successfully. Across all targets, GPT reached 18.90% branch coverage and 16.21% mutation score. The dual threshold was reached by 13/63 functions. On the executable paired subset, no GPT–EvoSuite difference remained statistically significant after Holm correction. The result highlights execution validity as the main constraint in this zero-shot setting and separates aggregate capability from pass-conditioned paired comparisons.

Keywords: Large language models · Unit test generation · Mutation testing · Branch coverage · EvoSuite · Empirical software engineering

1 Introduction

Unit tests are a practical safeguard against regressions, but writing them manually requires developers to identify representative inputs, boundaries, and expected outcomes. Code-oriented language models make direct test generation possible: a model can receive a function and produce a JUnit class in one call. The HumanEval benchmark established a widely used controlled environment for evaluating such code-generation capability [16]; however, test generation carries a stricter requirement than code synthesis. A generated test must be executable and its assertions must agree with the system under test (SUT). A plausible-looking response returned by an API is therefore only the first stage of the evaluation, not evidence of test quality.

This study evaluates whether a fixed zero-shot prompt to GPT-4o-mini can generate useful JUnit 4 suites for Java functions. We draw 63 targets from a Java transformation of the HumanEval benchmark in which each original Python

function is re-implemented as a standalone Java class with a correct implementation and a manually introduced buggy variant; the correct variant is the SUT in this study. Unlike prior work that evaluates LLMs on the Python-based original benchmark [16], our setting targets JUnit 4 test generation for equivalent Java implementations and enforces a strict execution gate before any coverage metric is recorded. We report branch coverage and mutation score in parallel rather than relying on a single number. Branch coverage records whether decision outcomes were executed; mutation score records the proportion of artificially seeded faults killed by the suite. Together, they separate broad code traversal from fault-detection strength.

We compare GPT-generated suites against retained EvoSuite suites. EvoSuite is a mature search-based Java test-generation tool [12]; its archived 1-, 3-, and 5-minute suites were re-measured under the same Maven, JaCoCo, and PIT pipeline used for the GPT results. This is an operational technical comparison, not a student benchmark. The originally planned student-written comparator has no validated per-function measurements in the same pipeline and is therefore explicitly deferred.

The central research question is whether zero-shot GPT-4o-mini tests meet predefined coverage and mutation thresholds and how their metrics differ from EvoSuite on the same SUTs. We examine five subquestions: (RQ1) branch-coverage threshold attainment; (RQ2) mutation-score floor and target attainment; (RQ3) paired GPT–EvoSuite differences at each EvoSuite budget; (RQ4) simultaneous threshold attainment; and (RQ5) the technical failure patterns that block valid execution.

This paper makes three contributions: (1) a fully traceable execution report for 63 Java targets, including raw API usage logs, repair records, and class-level coverage and mutation metrics; (2) a paired, budget-specific comparison with existing EvoSuite suites using a rigorous statistical design with Holm correction; and (3) an explicit analysis boundary that prevents pass-conditioned results from being presented as representative of all generated suites. The replication package retains the prompt template, generation scripts, raw result files, and analysis notebook needed to reproduce every reported figure.

2 Background and Related Work

Automated test generation has long used search-based techniques. EvoSuite generates Java unit tests and assertion stubs while optimizing an objective such as branch coverage [12]. It is consequently a natural technical reference point for a Java experiment, though its performance should not be interpreted as a proxy for human-written tests.

Language-model evaluation introduced code-generation benchmarks such as HumanEval [16]. The Java corpus used in this study transforms that setting into standalone Java classes, each with a correct implementation (the SUT) and a buggy variant. In contrast to program synthesis, test generation must construct an executable oracle. A syntactically valid assertion can still cause the test suite

to fail when the expected value does not match the SUT's actual output; this motivates reporting compile and pass status as a prerequisite before interpreting coverage or mutation metrics.

The study measures branch coverage using JaCoCo [9] and mutation score using PIT [5]. These metrics answer distinct questions. A suite can traverse every decision branch while still accepting incorrect return values if its assertions are weak or incorrectly specified. Wang et al. demonstrate this precisely: in the mutation-guided generation setting, a test suite can reach 100% line coverage while killing fewer than 4% of seeded mutants, confirming that coverage alone is a poor proxy for fault-detection strength [14]. Mutation score is therefore reported alongside branch coverage throughout this paper rather than being treated as a secondary metric.

Our inferential design follows standard guidance for statistical comparisons in randomized software engineering experiments [13]. The RQ3 comparisons are paired by SUT, two-sided, and evaluated independently for each EvoSuite budget and metric combination. The six resulting p-values are adjusted with the Holm step-down procedure to control the family-wise error rate. This design preserves the per-SUT pairing rather than averaging across unrelated populations.

The coverage and mutation thresholds used in this study are drawn from an external benchmark. Huang et al. report mean branch coverage of 30.22% and mean mutation score of 40.21% across 12 LLMs on the 3,909-function ULT benchmark, a data-leakage-free evaluation corpus of real-world functions [6]. We adopt those values as reference targets while acknowledging the differences in programming language and benchmark composition. The 4.00% mutation floor is a deliberately low operational screen derived from the warning case in Wang et al. [14]: it flags suites that show essentially no fault-detection signal without imposing an unrealistic quality bar.

2.1 LLM Test Generation on Java Benchmarks

Several studies address LLM-based test generation for Java in settings directly comparable to ours. Yuan et al. evaluated Codex, GPT-3.5-Turbo, and StarCoder on both the HumanEval and the EvoSuite SF110 benchmarks, finding that while Codex achieved above 80% coverage on the simpler HumanEval tasks, no evaluated model exceeded 2% coverage on the harder SF110 benchmark [16]. This benchmark-dependency effect contextualises our use of HumanEval-Java as a controlled starting point rather than a representative industrial setting.

The AgoneTest framework provides the most directly comparable configuration to ours: it evaluated `gpt-4o-mini` in zero-shot mode on a large Java corpus, reporting 41.9% branch coverage and 44.5% mutation score for valid suites [11]. That study also notes low initial compilation success rates, consistent with the execution-validity limitation we observe. Yang et al. found that default ChatGPT test suites suffer from compilation errors, execution failures, and incorrect assertions at a notable rate, while iterative feedback-based repair significantly improved assertion correctness [15]—a result that motivates the repair step included in our protocol and the follow-up designs discussed in Section 5.

For mutation-specific evaluation on Java, Lima et al. applied GPT-4o to six Defects4J classes and reported a mean mutation score of 87.11% across runs that produced valid PIT reports [8]. The authors note that not every model–class combination yielded a valid report, mirroring the execution-gate issue in our setting. The Test Wars study compared LLM-based generation (TestSpark with GPT-4o) against EvoSuite and symbolic execution on GitBug Java, finding that LLMs produced lower branch coverage than EvoSuite but higher mutation scores on the same targets [4]. That pattern is consistent with the pass-conditioned subset results in our RQ3 analysis.

A broader large-scale study of prompt engineering for LLM test generation across 216,300 generated test cases on Defects4J and other Java benchmarks found compilation failure rates as high as 86% under some prompt configurations and reported persistent hallucination-driven failures including non-existent API calls and fabricated dependencies [10]. The prevalence of assertion failures in our results—47 of 49 invalid suites—aligns with this pattern, though the specific failure mode (incorrect oracle values rather than missing symbols) differs slightly.

2.2 Why Execution Status Must Be Reported

An executable test suite is a prerequisite for interpreting any test-quality metric. A test class may fail to reach the SUT because it references a non-existent method, contains a logically incorrect expected value, or uses an unsupported JUnit API. Treating these outcomes as minor formatting issues would overstate an LLM's practical usefulness as a test generator. We therefore maintain three distinct layers of evidence: generation status records whether the API returned content; execution status records whether Maven successfully compiled and ran the suite; and quality metrics are recorded only after a suite clears the execution gate.

This layered separation is especially consequential for paired comparisons. If only the GPT suites that pass are included in the RQ3 analysis, the estimated difference addresses a conditional question—how do suites that survived validation compare with EvoSuite?—rather than characterising the expected output of the generation pipeline as a whole. The full-corpus and pass-conditioned views therefore appear side by side throughout this paper so that neither masks the other.

2.3 Comparator Scope

EvoSuite is a well-established technical baseline for Java test generation because it produces complete, executable suites under a specified search budget [7]. Using three explicit time budgets (1, 3, and 5 minutes) makes the comparator configuration transparent and reproducible. However, an automatic search-based suite and a student-written suite are answers to different empirical questions, and neither should be substituted for the other. The present comparator evaluates the GPT protocol against EvoSuite on identical SUTs; it does not make any claim about student capability or simulate student behaviour.

3 Methodology

3.1 Subjects and Protocol

The study corpus consists of 63 Java functions drawn from a local transformation of the HumanEval benchmark [16]. Each original Python function from HumanEval was re-implemented as a standalone Java class containing a correct implementation (the SUT) and a manually introduced buggy variant; functions with degenerate or trivially simple branch structure were excluded during corpus assembly. The correct implementation is the test target throughout this study. Prior work using the same benchmark in the Python setting, such as Yuan et al. [16], provides a useful point of reference, though the Java compilation and assertion requirements make direct score comparison between languages inappropriate.

For each target, the generation script supplied its complete source code to `gpt-4o-mini-2024-07-18` with a fixed zero-shot instruction. The prompt required a single executable JUnit 4 class in package `humaneval.correct`, correct assertions, and coverage of normal, boundary, and error cases. Temperature was fixed to zero. No output-token cap was set in the API call.

The run retained a timestamped API log and a CSV record for each target. The initial call was retried only for retryable service failures. One subsequent repair invocation was permitted for syntactic or assertion issues. Generated tests were then compiled and executed with Maven. Tests that remained invalid were archived rather than deleted, and their execution status was retained in the result set.

The comparator consists of existing EvoSuite archives generated under 1-, 3-, and 5-minute budgets. No EvoSuite generation was performed for this study. Each archive contains one suite per SUT and was measured against the same target set with the same Maven, JaCoCo, and PIT configuration. Temurin JDK 8 was used for this measurement because EvoSuite 1.0.6 requires `tools.jar`.

3.2 Evidence Retention and Postprocessing

The pipeline records a row per SUT for generation, repair, execution status, and class-level metrics. The primary files are the full API-output CSV, repair-output CSV, compile-status CSV, GPT class-metrics CSV, three EvoSuite class-metrics CSVs, a measurement manifest, and the derived summary CSV. The notebook reads these retained records rather than manually copying values into a report. A validation script checks the number of distinct SUTs, status partition, coverage numerators, mutation numerators, comparator pass counts, RQ3 rows, notebook outputs, and figure resolution.

Postprocessing does not repair the scientific record silently. Generated test classes are moved into timestamped archive or invalid-test locations after measurement rather than deleted. The active test directory is cleared only after preservation. This matters because a later reader must be able to distinguish a pipeline configuration change from a changed test suite. The paper reports the

initial API run and the permitted repair pass separately, so the final execution gate is not confused with a single unaided model response.

3.3 Measures and Decision Rules

For every SUT, JaCoCo [9] produced branch-covered and branch-total counts; branch coverage is the covered-to-total ratio. PIT [5] produced killed-mutant and eligible-mutant counts; mutation score is their ratio. A suite was considered executable only if its test class compiled and passed all test cases without modification. The report retains zero or unavailable class-level outcomes for non-executable suites, while the paired RQ3 analysis includes only SUTs that passed in both tools.

RQ1 tests whether the per-SUT GPT branch score exceeds 30.22%. RQ2 tests the mutation score against an empirical 4.00% floor and a 40.21% target. RQ4 tests whether more than half of the 63 SUTs simultaneously pass, reach branch coverage of at least 30.22%, and reach mutation score of at least 4.00%. RQ1 and RQ2 use one-sided Wilcoxon signed-rank tests; RQ4 uses an exact binomial test. The significance level is $\alpha = 0.05$.

For RQ3, GPT and EvoSuite scores are paired by SUT for branch coverage and mutation score at each budget. The test is a two-sided paired Wilcoxon signed-rank test. Three technical mutation exclusions are omitted from mutation-score tests. The six p-values are Holm-adjusted. We report the raw and adjusted p-values, the paired and non-zero-ranked sample sizes, percentage-point difference, and rank-biserial effect size, following standard evaluation guidelines [13]. A comparison is considered statistically significant only when the adjusted p-value is below 0.05.

The report preserves two denominators for RQ3. The paired denominator counts SUTs that executed in both tools. The ranked denominator counts non-zero differences used by the signed-rank statistic. When many pairs have identical scores, the latter can be much smaller. Reporting both prevents a small ranked sample from being mistaken for 63 independent observations.

3.4 Generation Prompt (Verbatim Template)

The following is the exact template passed as the user message. The two placeholders were instantiated separately for each SUT; `{source_code}` was replaced by the complete source of that target. No example test, previous model response, or few-shot demonstration was included.

> You are an expert Java developer and software tester. Your task is to write a comprehensive JUnit 4 test suite for the following Java class. Strictly adhere to the following requirements:
> 1. Generate test cases using JUnit 4 (use org.junit.Test, org.junit.Assert).
> 2. Do not use JUnit 5 or other test frameworks.
> 3. Test all logical paths, edge cases, boundary values, and potential error conditions.

4. Ensure all assertions are correct and correspond exactly to the expected behavior of the correct code.
5. The test class must be named `{class_name}_GPTTest`, corresponding to the target class `{class_name}`.
6. The test class must be in package `humaneval.correct`.
7. You should import any required packages (like java.util.*).
8. Provide only the executable Java test class code. Do not include any markdown explanations, text wrapping, or extra commentary.
Source Code: `{source_code}`

3.5 Reproducibility and Protocol Amendment

The analysis notebook, derived CSVs, figures, API usage records, execution statuses, archive manifest, and validation report are stored in the replication package. The original proposal motivated a future comparison with student-written tests. Comparable per-SUT student measurements were unavailable, so that comparison is deferred. The present paper consequently makes no numerical claim about student performance and never treats EvoSuite as a student proxy.

4 Results

4.1 Execution Validity (RQ5)

The initial API generation returned content for all 63 targets, with an initial recorded cost of $0.030982. A single repair pass recorded 55 repair calls costing $0.036773. After compilation and execution, 14 of the 63 GPT suites passed. Forty-seven suites failed assertions and two failed compilation. Thus, assertion failures accounted for 47 of the 49 invalid suites (95.9%), making execution-valid oracle construction the predominant observed limitation.

4.2 Threshold Results (RQ1, RQ2, and RQ4)

Across all 63 targets, GPT reached 144/762 covered branches (18.90%) and killed 135/833 eligible mutants (16.21%). Table 1 reports the predefined hypothesis tests with the effect size and analysis denominator for each RQ. RQ1 was not supported. The aggregate mutation score exceeded the 4.00% floor, but the per-SUT one-sided test did not support RQ2 at either the floor or target. For RQ4, only 13/63 SUTs simultaneously passed and met both thresholds (20.63%); the exact binomial result did not support a majority.

For RQ1 and RQ2, r_{rb} is the rank-biserial effect of the per-SUT difference from the stated threshold; negative values indicate that the ranked differences favor values below the threshold. For RQ4, RD is the observed dual-success rate minus the hypothesized 50% majority rate. RQ5 is descriptive rather than an inferential hypothesis: its denominator is all 63 targets, and it reports 14 passing suites, 47 assertion failures, and two compilation failures.

Fig. 1. Execution status of all 63 GPT-generated suites.

Table 1. Threshold and dual-success results.

RQ	Test	N	Aggregate/Rate	p	Effect	Decision
RQ1	Branch $> 30.22\%$	63	18.90%	0.942967	$r_{rb} = -0.215$	Not supported
RQ2 floor	Mutation $> 4.00\%$	60	16.21%	0.952447	$r_{rb} = -0.233$	Not supported
RQ2 target	Mutation $> 40.21\%$	60	16.21%	0.997231	$r_{rb} = -0.387$	Not supported
RQ4	Dual success $> 50\%$	63	13/63 (20.63%)	0.9999996	$RD = -29.37$ pp	Not supported

4.3 Paired Comparison with EvoSuite (RQ3)

All retained EvoSuite suites passed for each budget. Its aggregate branch coverage rose from 90.29% at one minute to 99.34% at five minutes; mutation score rose from 73.23% to 82.11%. The paired analysis is deliberately narrower: it is conditioned on the 14 GPT suites that also passed. This selection is why the pass-conditioned GPT means in Table 2 should not be read as full-set GPT performance.

No RQ3 comparison was significant after Holm correction. The smallest adjusted p-value was 0.375053 for the five-minute mutation comparison. Figure 3 visualizes the paired measurements.

For branch coverage, only two of the 14 paired SUTs had non-zero score differences; the remaining 12 pairs produced identical scores for both tools. The signed-rank statistic is therefore based on $N_r = 2$ ranked differences, which gives the branch test essentially no statistical power. The non-significant result for branch coverage should be read as an artifact of near-total agreement within this specific pass-conditioned subset rather than as evidence that GPT and EvoSuite branch scores are equivalent in general. The mutation comparisons, by contrast,

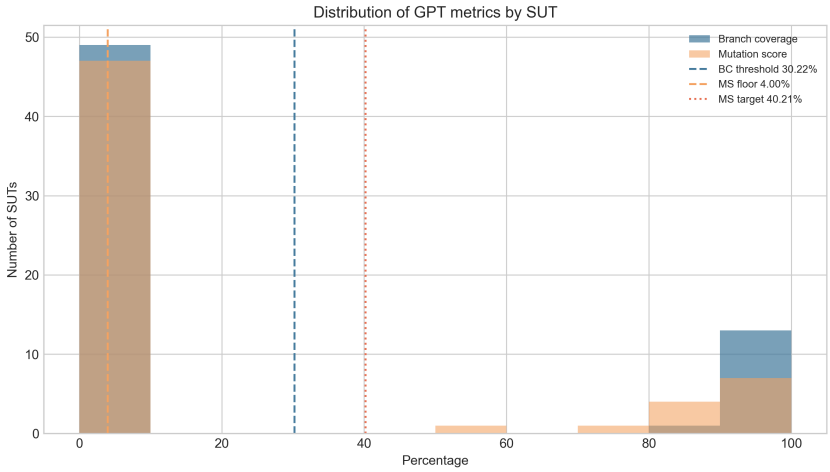

Fig. 2. Distribution of GPT branch coverage and mutation score across the full SUT set.

Table 2. Paired GPT–EvoSuite results. N_p/N_r denotes paired/non-zero-ranked SUTs; Δ is GPT minus EvoSuite in percentage points.

Budget	Metric	N_p/N_r	GPT mean	Evo mean	Δ	Raw p	Holm p	r_{rb}
1 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
1 min	Mutation	13/7	88.21%	79.14%	+9.07	0.204084	0.898562	+0.536
3 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
3 min	Mutation	13/7	88.21%	78.20%	+10.01	0.236724	0.898562	+0.500
5 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
5 min	Mutation	13/7	88.21%	82.40%	+5.81	0.062509	0.375053	+0.786

ranked seven non-zero pairs and produced more informative effect sizes, though none reached significance after correction.

The full EvoSuite aggregates provide a separate descriptive baseline. At one minute, EvoSuite covered 688/762 branches (90.29%) and killed 610/833 mutants (73.23%). At three minutes, the corresponding values were 730/762 (95.80%) and 638/833 (76.59%). At five minutes, they were 757/762 (99.34%) and 684/833 (82.11%). All 63 archived suites passed at every listed budget. These totals demonstrate the technical consistency of the retained comparator; they are not included in the paired hypothesis-test denominator unless the GPT counterpart also passed.

The observed difference between aggregate and pass-conditioned GPT scores is material. The full corpus includes every generated suite, including the 49 that did not pass. The RQ3 mean branch score of 98.09% and mutation score of 88.21% therefore characterize a selected subset, not the overall generator. This is why Tables 1 and 2 answer complementary questions rather than conflicting ones.

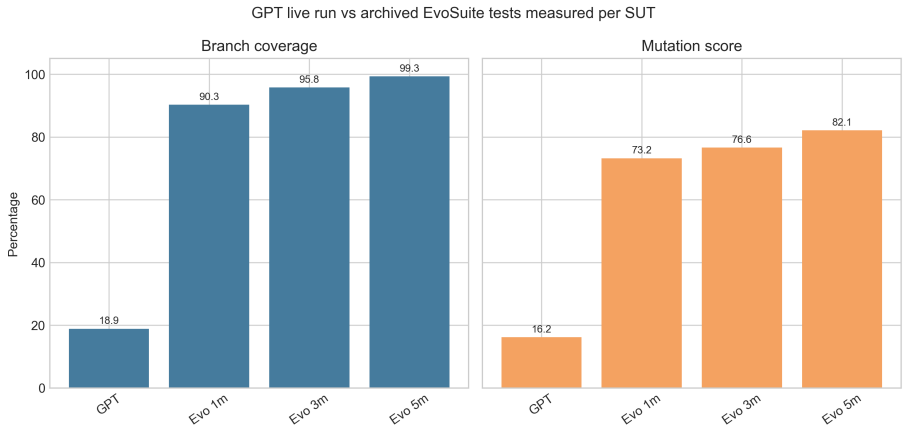

Fig. 3. Pass-conditioned paired GPT–EvoSuite comparison.

5 Discussion

The primary finding is the gap between generation availability and executable test quality. GPT returned content for every target, yet 49 suites remained invalid after the allowed repair step. This makes the full-set branch coverage and mutation score modest, and it prevents the predefined RQ1, RQ2, and RQ4 hypotheses from being supported. The result is useful even though it is negative: in this protocol, zero-shot generation did not consistently produce valid test oracles for the chosen Java targets.

The paired results require a different interpretation. When both GPT and EvoSuite suites pass, the selected GPT suites have high coverage and mutation scores, and the five-minute mutation comparison has the smallest raw p-value. However, the paired sample contains only 14 executable GPT suites (13 for mutation) and the branch analysis has two non-zero ranked differences. After controlling the six RQ3 comparisons with Holm adjustment, no difference is significant. These results do not establish parity between tools; they establish that this run lacks statistically significant evidence of a difference on the small pass-conditioned subset.

For practice, the observed workflow suggests that an LLM can be a useful candidate-test generator only when followed by strict compilation, execution, and oracle validation. A raw response should not be counted as a successful test suite. Search-based generation remained consistently executable on the archived EvoSuite suites under the measured budgets, while the LLM pipeline's main improvement opportunity is repair or feedback targeted at assertion validity.

The study also clarifies the scope of the comparator. A separate feasibility evaluation carried out prior to this run confirmed that EvoSuite branch coverage reaches a plateau at 99.61% for budgets beyond 30 minutes on this corpus [7]; the three remaining uncovered branches are structurally unreachable

by the search regardless of budget. The 1-, 3-, and 5-minute budgets used in RQ3 therefore span a meaningful range of EvoSuite performance—rising from 90.29% to 99.34% branch coverage—while stopping well below the point of diminishing returns. EvoSuite supplies a reproducible engineering baseline on the same SUTs; it does not measure what students write. A future study should collect student-written suites and evaluate them with the identical build, coverage, and mutation pipeline before making any GPT–student claim.

5.1 Practical Implications

For a developer or teaching team, the immediate implication is not to reject LLM assistance altogether. Instead, the generation step should be treated as a proposal mechanism. A safe workflow keeps the generated source, compiles it in isolation, executes it against the known-correct SUT, and measures it only after it passes. A failure category should remain visible in the final dashboard. This workflow prevents a high-level language-model success rate from obscuring the cost of manual debugging or incorrectly asserted behavior.

The result also motivates feedback-guided repair. The present run allowed only one repair invocation and did not use coverage or mutation feedback to steer subsequent generation. Because assertion failures dominate the observed invalid outcomes, a next protocol can give the model the compiler output, failing assertion, or minimized counterexample and then evaluate whether a bounded repair policy raises the executable-suite rate without inflating API cost [1, 2]. Such a follow-up should predefine the maximum repair attempts and report the separate contribution of each attempt.

5.2 Interpretation of Negative Findings

The negative threshold results should not be described as evidence that LLMs cannot generate useful tests. They establish a narrower fact: this model snapshot, fixed prompt, and zero-shot repair policy did not meet the preregistered thresholds on this 63-SUT setting. The evidence is informative because it identifies which condition failed most often. A future comparison can alter one factor at a time–for example, structured few-shot prompting, test-feedback loops, or domain-specific context [3]–while retaining the same execution and metric gates.

5.3 Measurement and Reporting Implications

This evaluation also demonstrates why a single aggregate number is insufficient for an LLM testing study. The all-target scores answer a deployment-oriented question: if a team runs the configured process once for every selected function, what quality is obtained without discarding failures? The pass-conditioned paired analysis answers a different diagnostic question: when both approaches produce runnable suites for the same target, is there evidence that their measured adequacy differs? Mixing the denominators would hide the practical failure

rate in the former question or unfairly attribute failures to a tool in the latter question. The report therefore keeps the 63-target population, the executable GPT subset, the paired subset, and the non-zero ranked subset explicit in both prose and tables.

The use of mutation analysis has a similar implication. Branch coverage indicates whether tests traverse decisions, whereas mutation score asks whether their assertions expose seeded behavioral changes. A generated suite can execute many branches while still accepting incorrect values, especially when assertions are weak, over-general, or based on an incorrect interpretation of the specification. Conversely, a small test suite may kill a high proportion of the mutants reachable in a simple function. Reporting the two measures together does not make them interchangeable; it makes their distinct limits visible. For this reason, future extensions should retain the class-level counts behind each percentage and should classify the major invalid-suite modes, rather than only reporting a mean coverage score.

Finally, the study treats reproducibility as part of the result rather than a later packaging task. The retained generation records make it possible to rerun the computations, inspect a particular invalid class, and distinguish a model-output problem from a build or instrumentation problem. This is particularly important for API-based experiments because model aliases, service behavior, and local dependencies can change over time. A replication should preserve the dated model identifier, prompt template, retry and repair policy, target manifest, compiler version, metric-tool versions, and exact decision rules before comparing a new run with this baseline.

5.4 Operational Cost and Selection Effects

The recorded service cost is small for this run, but cost should not be interpreted independently of validity. The initial calls cost $0.030982 and the permitted repair calls cost $0.036773. These figures describe only the API portion of the workflow. They omit the local time needed to compile, diagnose a failure, measure the suite, and decide whether further human intervention is acceptable. In a practical setting, a comparison between test-generation methods should therefore report at least three dimensions: generation expenditure, executable-suite yield, and post-generation engineering effort. A cheap response that fails its assertions may still be expensive when it creates review work for a developer.

The RQ3 subset creates a second, statistical form of selection. By definition, it excludes the 49 GPT suites that did not execute. This is appropriate for a paired measurement of score differences because coverage and mutation scores cannot be compared fairly when one side has no valid test class. It is not appropriate as a stand-alone claim about end-to-end test generation. The full-set and paired views should consequently be read together. The former bounds the outcome a user receives from the configured pipeline; the latter describes the quality of the successful cases. Presenting both avoids the common but misleading practice of reporting high coverage only after silently filtering failed generations.

5.5 Design for a Follow-Up Study

A direct follow-up should change as few variables as possible while targeting the observed bottleneck. One straightforward design holds the 63-SUT corpus and the JaCoCo/PIT measurement configuration constant and varies the prompt strategy: fixed zero-shot, structured few-shot, and bounded feedback-guided repair. Each condition should use the same dated model identifier, the same maximum repair attempts, the same temperature, and the same token-accounting policy. The primary reported outcome should remain the executable-suite rate, with branch and mutation scores reported separately for the full corpus and the passing subset. Such a design would directly answer whether additional context or iterative feedback improves the rate of valid oracle construction under otherwise identical experimental conditions.

6 Threats to Validity

Internal validity. The protocol fixed the model snapshot, prompt, temperature, target sources, and measurement tooling. Nevertheless, model-service behavior and repair outcomes can vary over time. The retained API usage records, raw responses, and timestamps mitigate but do not remove this risk.

Construct validity. Branch coverage and mutation score are proxies for test quality, not complete definitions of it. We mitigate single-metric bias by reporting both measures and execution status. Non-executable suites are not treated as valid evidence of coverage quality.

Conclusion validity. RQ3 is conditional on successful execution, resulting in 14 branch pairs and 13 mutation pairs. The number of non-zero branch differences is particularly small. We report raw and Holm-adjusted p-values, effect sizes, and both paired and ranked sample sizes, but the study has limited power for this subset.

External validity. The 63 Java functions come from one transformed benchmark and one fixed zero-shot model configuration. The findings should not be generalized to other models, prompting strategies, repositories, languages, or student populations without further measurements. In particular, the absence of validated student per-function data means no conclusion about student-written tests is supported.

Measurement validity. The aggregate score denominators are produced by the configured JaCoCo and PIT runs. Three technical mutation exclusions are removed only from the appropriate Wilcoxon samples and are stated in the summary CSV. The project retains exact numerators and class-level rows so that a future change in tool version or inclusion rule can be detected. The study does not claim that these two metrics capture readability, maintainability, or developer effort.

Protocol deviation. The original motivation included a student-written benchmark. It is not substituted with EvoSuite in this paper. The operational amendment limits the technical comparison to retained EvoSuite archives and

records instructor confirmation as pending. This disclosure reduces the risk that an implementation convenience is later reported as a completed human-subject comparison.

7 Conclusion

We evaluated zero-shot GPT-4o-mini test generation on 63 HumanEval-Java functions using execution status, branch coverage, mutation score, and paired EvoSuite comparisons. GPT produced an API response for all 63 targets, but only 14 suites were executable after one permitted repair pass. Across the full corpus, branch coverage was 18.90% and mutation score was 16.21%; only 13 functions achieved both thresholds simultaneously. RQ1, RQ2, and RQ4 were not supported. For RQ3, no GPT–EvoSuite comparison reached significance after Holm correction on the pass-conditioned subset. Assertion failures accounted for 95.9% of the 49 invalid suites, identifying oracle construction as the primary obstacle in this zero-shot protocol.

Future work should pursue two directions. First, the prompt strategy should be varied systematically—comparing zero-shot, few-shot, and feedback-guided repair—while holding the SUT corpus and measurement pipeline constant, to determine whether additional context reduces the assertion-failure rate without inflating API cost. Second, the deferred student comparison requires its own protocol: student-written test suites should be collected with consent, compiled and executed in the same Maven environment, and measured without modifying their assertions. Crucially, the study must preregister how incomplete or non-compiling submissions are handled, how multiple attempts per student are counted, and whether the comparison unit is a student, a suite, or a function. Only once those choices are fixed and documented can an LLM–student statement be statistically and ethically sound. Until then, EvoSuite remains the appropriate reproducible search-based baseline, and no claim about student-written test quality should be inferred from either tool's results.

References

1. LLMLOOP: Improving llm-generated code and tests through automated iterative feedback loops. In: 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME) (2025). https://doi.org/10.1109/ICSME64153.2025.00109
2. A three-stage pipeline using react and reflexion for reliable llm-based java unit test case generator. In: 2025 IEEE International Conference on Software Testing, Verification and Validation (ICST) (2025), `https://ieeexplore.ieee.org/abstract/document/11351825/`
3. What inputs drive effective large language model-based unit test generation? IEEE Software (2025). https://doi.org/10.1109/MS.2025.3621625
4. Alkhateeb, F., Kochhar, P.S., Gorla, A.: Test Wars: A comparative study of SBST, symbolic execution, and LLM-based approaches to unit test generation. In: 2025 IEEE International Conference on Software Testing, Verification and Validation (2025). https://doi.org/10.1109/ICST62969.2025.10989033, `https://doi.org/10.1109/ICST62969.2025.10989033`

5. Bouafif, M.S., Hamdaqa, M., Zulkoski, E.: PRIMG: Efficient llm-driven test generation using mutant prioritization. In: Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering. pp. 1107–1116. ACM (2025). https://doi.org/10.1145/3756681.3756991, http://dx.doi.org/10.1145/3756681.3756991
6. Huang, D., Zhang, J.M., Harman, M., Zhang, Q., Du, M., Ng, S.K.: Benchmarking LLMs for unit test generation from real-world functions. ACM Transactions on Software Engineering and Methodology (2026). https://doi.org/10.1145/3805043, https://doi.org/10.1145/3805043
7. Lemieux, C., Inala, J.P., Lahiri, S.K., Sen, S.: CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). pp. 919–931. IEEE (2023). https://doi.org/10.1109/icse48619.2023.00085, http://dx.doi.org/10.1109/icse48619.2023.00085
8. Lima, M., Santos, A., de Souza, S.: Evaluating LLM-generated unit tests with mutation testing: ChatGPT vs DeepSeek. In: Anais do XXIV Simpósio Brasileiro de Qualidade de Software (2025). https://doi.org/10.5753/sbqs.2025.13792, https://doi.org/10.5753/sbqs.2025.13792
9. Lu, Z., Zhang, P., Nie, Y., Yang, Y., Tang, Y., Chong, C.Y., Zhou, Y.: Beyond coverage: Automatic test suite augmentation for enhanced effectiveness using large language models. Proceedings of the ACM on Programming Languages **10**(OOPSLA1), 1403–1431 (2026). https://doi.org/10.1145/3798251, http://dx.doi.org/10.1145/3798251
10. Nie, P., Banerjee, R., Li, J.J., Mooney, R.J., Gligoric, M.: Prompt engineering in LLMs for automated unit test generation: A large-scale study. Empirical Software Engineering (2026). https://doi.org/10.1007/s10664-026-10840-4, https://link.springer.com/article/10.1007/s10664-026-10840-4
11. Schäfer, M., Dixit, N., Rajbhandari, P., Lal, A.: LLMs for automated unit test generation and assessment in Java: The AgoneTest framework (2025), https://doi.org/10.48550/arxiv.2511.20403
12. Tang, Y., Liu, Z., Zhou, Z., Luo, X.: ChatGPT vs SBST: A comparative assessment of unit test suite generation. IEEE Transactions on Software Engineering **50**(6), 1340–1359 (2024). https://doi.org/10.1109/tse.2024.3382365, http://dx.doi.org/10.1109/tse.2024.3382365
13. Wang, C., et al.: On the evaluation of large language models in unit test generation (2024), https://arxiv.org/html/2406.18181v2
14. Wang, G., Xu, Q., Briand, L., Liu, K.: Mutation-guided unit test generation with a large language model. IEEE Transactions on Software Engineering **52**(5), 1657–1671 (2026). https://doi.org/10.1109/TSE.2026.3682975, https://doi.org/10.1109/TSE.2026.3682975
15. Yang, Q., Zhao, C., Liu, W., Chen, J., Xu, C.: Evaluating and improving ChatGPT for unit test generation. In: Companion Proceedings of the ACM on Software Engineering. vol. 1 (2024). https://doi.org/10.1145/3660783, https://doi.org/10.1145/3660783
16. Yuan, Z., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X., Lou, Y.: Using large language models to generate JUnit tests: An empirical study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (2024). https://doi.org/10.1145/3661167.3661216, https://doi.org/10.1145/3661167.3661216